NGUYEN DEVELOPMENT · PHPDBADMIN

Deployment — phpdbadmin

Umgebungen, Pipeline, Betrieb, Testmanagement, Release-Notes

Phase	05 Deployment
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	30.10.2026, 10:58 · baut auf Commit 304a542 + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. 05 — Deployment

Inhalt

05 — Deployment

Installation beim Anwender

Betrieb und Sicherheit

Release veröffentlichen

Website

Pipeline

05 — Deployment

> **PDF der Phase:** deployment.pdf — bei jeder Änderung neu gebaut (`python3 dokumentationen/werkzeuge/pdf-bauen.py --nur 05`).

Stand 30.10.2026: v1.3.0 veröffentlicht, mit Web-Installer; Landingpage gebaut, noch nicht veröffentlicht; v1.0 ist installierbar (B-05, TB 08). Zielbild nach R-11 (lokal und selbst gehostet) und R-30:

Stufe	Ort	Bemerkung
Dev	`https://phpdbadmin.test`	Laravel Herd (macOS), PHP 8.5; MySQL 8.4 :3307, MariaDB 10.11 :3306, PostgreSQL 18 :5432
Demo	offen	öffentliche Demo mit SQLite-Beispieldatenbanken (cyon oder Laravel Cloud) — Entscheid Hoang
Produktion	beim Anwender	selbst gehostet neben den eigenen Datenbanken
Website	noch nicht veröffentlicht	statische Landingpage aus `website/` (Deutsch, Englisch) — Ort offen: GitHub Pages des Repositorys oder eigene Domain, Entscheid Hoang

Installation beim Anwender

Die Anleitung steht im README des Repositorys.

Web-Installer (seit v1.3.0): `phpdbadmin-installer.php` aus dem Release in den leeren Ordner einer eigenen (Sub-)Domain legen und öffnen. Eine Datei ohne Abhängigkeiten (PHP-7.3-Syntax, damit ein zu altes PHP eine Erklärung zeigt); Quelle `resources/installer/phpdbadmin-installer.php`, Tests `tests/Feature/Installer/WebInstallerTest.php` (echter PHP-Server: Einrichten, verändertes Archiv, falsche und fehlende Signatur, Pfad aus dem Archiv hinaus, zweiter Browser, Reihenfolge der Schritte, fremde Dateien). Ablauf:

Schritt	Was geschieht
Prüfen	Tatsachen messen (PHP, Erweiterungen, Treiber, Speicher, Platz, HTTPS, Webserver, Ordner, Unterordner-Adresse), Hoster erkennen (Rechnername, Pfad, Kennzeichen; Profile aus der Recherche für Basalto, Stand 28.10.2026), Befunde mit Schritten beim Hoster und Support-Nachricht
Selbsthilfe	PHP per `.htaccess` (Testordner, vom Browser abgefragt, erst dann übernommen, Sicherung `.htaccess.phpdbadmin-vorher`), fremde Dateien nach `.phpdbadmin-vorher-<zeit>`, Datenbank-Server durchprobieren (`localhost` ↔ `127.0.0.1`, Hoster-Namen), Datenbank anlegen, Test-Mail per SMTP
Herunterladen	`latest.json` und ZIP von GitHub (`releases/latest/download`) oder aus dem Ordner; Grösse, SHA-256 und Ed25519 über `phpdbadmin-release:1\nversion\ndatei\ngrösse\nsha256`
Einrichten	Entpacken mit Pfadprüfung, `.env` (Schlüssel neu, Adresse aus dem Browser, Datenbank, Mail, erlaubte Hosts; Rechte 600), Programm nach `~/phpdbadmin-app/<host>` (sonst `.phpdbadmin` im Webordner, gesperrt), Inhalt von `public/` in den Webordner, `index.php` mit `usePublicPath`, `.htaccess` mit Sperre für Punkt-Dateien, Brücke `bootstrap/phpdbadmin-install.php` (Migrationen, erstes Konto, Einrichtungslink, Systemprüfung)
Abschliessen	vom Browser aus `/up` und (Programm im Webordner) die Sperre prüfen; Zwischenstände, Zustand und Installer löschen; Einrichtungslink und Cron-Zeilen zeigen

Nur ein Browser bedient den Installer (Token im Cookie und im Formular, frei nach 30 Minuten ohne Aktivität). phpdbadmin läuft nur direkt auf einer (Sub-)Domain, nicht in einem Unterordner (Pfade der Oberfläche sind absolut).

Mit Konsole:

1. `git clone https://github.com/hoang-79/phpdbadmin.git` (später `composer create-project nguyen-development/phpdbadmin`)
2. `composer setup` — Abhängigkeiten, `.env`, Oberfläche bauen, App-Datenbank, Systemprüfung, erstes Konto mit Einrichtungslink
3. `.env`: `APP_URL` (HTTPS für Passkeys), `APP_ENV=production`, `APP_DEBUG=false`, `MAIL_*`, `PHPDBADMIN_ALLOWED_HOSTS`, `PHPDBADMIN_SQLITE_PATHS`, `SESSION_SECURE_COOKIE=true`
4. Cron `* * * * * php artisan schedule:run` (Aufräumen: Protokoll, Exporte, Codes, Einladungen) und ein Worker `php artisan queue:work --tries=1 --timeout=3600` als Dienst
5. `php artisan phpdbadmin:check` — zeigt, was fehlt

Ohne Konsole: Code hochladen, Dokumentenwurzel `public/`, Seite öffnen — die Ersteinrichtung verlangt den Einrichtungsschlüssel aus `storage/app/private/einrichtungsschluessel.txt` (T-19).

Aktualisieren: `git pull && composer upgrade` (Abhängigkeiten, Oberfläche, Migrationen, Systemprüfung). `APP_KEY` nie ändern — gespeicherte Passwörter der Verbindungen wären verloren.

Betrieb und Sicherheit

- **Header:** CSP mit Nonce je Antwort, `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: same-origin`, HSTS in Produktion über HTTPS (`App\Http\Middleware\SecurityHeaders`). In `APP_ENV=local` meldet die CSP nur (Herd fügt ein eigenes Skript ein).
- **Erlaubte Hosts:** Zielverbindungen nur zu `PHPDBADMIN_ALLOWED_HOSTS`; Datenbanknamen, die den DSN verändern würden, und die App-Datenbank selbst sind nie erreichbar (Sicherheitsbericht).
- **App-Datenbank:** am besten SQLite oder ein eigener Server bzw. ein eigenes Konto mit Passwort, das niemand für Zielverbindungen kennt — `phpdbadmin:check` warnt sonst.
- **KI-Hilfe (Q-08):** aus, bis `PHPDBADMIN_AI=true`; jedes Konto braucht dann seinen eigenen Schlüssel; gesendet werden nur Frage und Schema an `api.anthropic.com`.

Release veröffentlichen

Releases baut Hoang lokal (spart Actions-Minuten, der Signaturschlüssel verlässt den Mac nie):

```
npm run build
php artisan phpdbadmin:release --key=~/.ssh/phpdbadmin/release-ed25519.key
gh release create v1.3.0 storage/app/release/* --title "phpdbadmin 1.3.0" --notes-file …
```

Version in `config/phpdbadmin.php` (`version`). Das Archiv enthält nur, was zur Laufzeit nötig ist (`vendor` ohne Entwicklungspakete, gebaute Oberfläche), dazu `latest.json` (Grösse, SHA-256, Ed25519-Signatur, Mindest-PHP, Erweiterungen) und den Installer mit eingesetztem öffentlichem Schlüssel und Repository. Geheimschlüssel: `~/.ssh/phpdbadmin/release-ed25519.key` (Rechte 600, nie ins

Repository); öffentlicher Schlüssel `2wyjL2KumyQ6vlmUvcv4NydLkoxJXr8U8OG9B+iYz/M=`. Neuer Schlüssel nur mit `phpdbadmin:release --keygen=<pfad>` — ältere Installer prüfen dann nicht mehr gegen neue Releases.

Website

Die Landingpage liegt in `website/` und ist rein statisch: keine Cookies, kein Tracking, nichts von Dritten (Schriften, Bilder, Symbole liegen mit auf dem Server). Ein Python-Skript ohne Abhängigkeiten baut sie nach `website/dist/` (nicht im Repository):

```
python3 website/bauen.py                                  # Basis: GitHub Pages des
        Repositorys
python3 website/bauen.py --basis https://example.org/     # andere Adresse (canonical,
        hreflang, Sitemap, Link-Vorschau)
php -S 127.0.0.1:8765 -t website/dist                     # ansehen
```

Ordner	Inhalt
`website/seiten/de`, `…/en`	Startseite, Impressum bzw. Legal notice, Datenschutz bzw. Privacy, 404 (zweisprachig); Kopfblock mit `titel`, `beschreibung`, `pfad`, `gegenstueck`; `_kopf` / `_fuss` als Bausteine je Sprache
`website/vorlagen/seite.html`	Rahmen mit Meta-Angaben, Open Graph, hreflang, JSON-LD (`SoftwareApplication`) auf den Startseiten
`website/assets`	`site.css` (Farben aus dem Logo-Kit, hell und dunkel nach Systemeinstellung), `site.js` (Menü, Kopieren, Einblenden), Schriften, Fotos als WebP mit `masse.json`
`website/werkzeuge`	`fotos.mjs` nimmt die Bildschirmfotos in der Sichtprüfung auf (Chrome ohne Fenster), `bilder.py` macht daraus WebP in zwei Grössen und schneidet SQL-Editor, EXPLAIN, Struktur und Befehlspalette zu

Platzhalter in den Seiten: `{{ version }}` (aus `config/phpdbadmin.php`), `{{ link:/pfad/ }}`, `{{ symbol:name }}` (Lucide aus `node_modules`), `{{ foto:name | Alt-Text }}`, `{{ foto-schema:name | … }}` (hell/dunkel), `{{ handy:name | … }}`. Die Vorlagen sind vom Formatierer ausgenommen (`vite.config.ts`), `website/**` löst keinen CI-Lauf aus. Fotos neu aufnehmen: Sitzungs-Cookie eines Kontos der Sichtprüfung als JSON nach `FOTO_SITZUNG`, dann `node website/werkzeuge/fotos.mjs` und `python3 website/werkzeuge/bilder.py /tmp/phpdbadmin-werkzeug/fotos`. Fotos mit lokalen Pfaden oder fremden Datenbanknamen (Verbindungsliste, Server-Übersicht) bleiben bewusst draussen.

Veröffentlichen: Inhalt von `website/dist/` hochladen (enthält `.nojekyll`, `404.html`, `robots.txt`, `sitemap.xml`). Impressum und Datenschutz nennen GitHub Pages als Hoster — bei einem anderen Ort beide Fassungen anpassen.

Pipeline

Regel «GitHub Actions sparsam»: je Push ein schlanker Auftrag (Pint, Larastan, vue-tsc, `vp check`, Tests auf SQLite); Matrix MySQL/MariaDB/PostgreSQL wöchentlich und von Hand; `paths-ignore` für Doku und Website, `concurrency` mit `cancel-in-progress`, `timeout-minutes`, Caches für Composer und npm. Seit 29.10.2026 auf github.com/hoang-79/phpdbadmin (öffentlich); erster Lauf mit dem Push von `main` und dem Tag `v1.3.0` (Release). Releases entstehen lokal (siehe oben), nicht in Actions. Beide Aufträge grün seit 29.10.2026 (Lauf). Was nur in der CI auffiel und behoben ist: SQLite-Arbeitsprozess

ohne die Testordner (Kindprozess sieht `config()` nicht), `localhost` mit `::1` unter Linux, Testkonten für `localhost` statt für die Adresse, unter der der Container den Runner sieht (`clientHost`), ANALYZE auf MySQL 8.0 (erst ab 8.3 angeboten), `pg_dump` 16 gegen PostgreSQL 18 (Workflow installiert die Werkzeuge in der Version des Servers) — und ein echter Fehler: **psql bis 14 schrieb mit `--single-transaction` fest, was vor einem abgelehnten Backslash-Befehl lief**; die Wiederherstellung setzt die Transaktion jetzt selbst (`BEGIN` / `COMMIT` in der Datei). Nachstellen lokal: Container auf den Untergrenzen (Colima), `PHPDBADMIN_TEST_<TREIBER>_PORT`.